Token Exhaustion Attacks: Treating LLM Consumption as a Security Boundary

Sebastian Wallace
BSc, BEng, CISSP, GWAPT, OSCP
October 2026. Paper 1 of the TEA series.

Abstract

Large language model (LLM) systems bill, throttle and fail in units of tokens. To that end we can then consider the token a security-relevant resource, yet token consumption is still handled mostly as a cost governance problem, through budgets, alerts and per-request caps. This paper proposes the Token Exhaustion Attack (TEA) as a security-first formalisation of deliberate resource exhaustion against LLM-backed systems, whether those systems face customers or are exercised by autonomous agents. It defines the threat, presents a taxonomy of seven attack vectors, and outlines a layered defensive architecture spanning input, output, model routing (AMS) and observability. The central claim is deliberately modest: TEA is not a newly discovered risk, but a risk that warrants the design discipline already applied to denial of service in conventional systems.

1. Introduction

A conventional denial of service attack needs no stolen credentials and no software flaw. It needs only that the target spends more on each request than the attacker does, and that nobody has decided where spending should stop. LLM-backed systems have inherited that economic shape in an unusually pure form. LLM-backed systems have inherited that economic shape in an unusually pure form. Every request is billed by the token, yet the cost of a response is only loosely tied to the size of the prompt: a few words can provoke thousands of tokens of output or hidden reasoning. Each of those tokens occupies GPU capacity, which is expensive and in limited supply, so wasted tokens cost the operator money and can crowd out legitimate users.

Current AI security effort is concentrated elsewhere. The dominant questions concern what a model says: harmful output, jailbreaks, prompt injection, and the leakage of sensitive data. These are serious problems, and they have attracted serious investment in guardrails. Consumption, by contrast, has been absorbed into the language of FinOps. The OWASP Top 10 for LLM Applications now lists Unbounded Consumption as a distinct risk and names denial of wallet within it (OWASP, 2025). The recommended controls, however, remain largely rate limits, quotas and spend alerts applied to individual requests.

This paper argues that this treatment stops short. A control that limits the size of one request says little about an adversary who sends ten thousand modest ones, or a single conversation engineered to grow, or a short prompt crafted to provoke an expensive chain of reasoning. The paper makes three contributions. It defines TEA as a security category with an explicit threat model (Section 3). It organises known and anticipated exhaustion techniques into a seven-vector taxonomy (Section 4). It sets out a layered mitigation architecture, including a lightweight complexity classifier that is designed so that it cannot itself be exhausted (Section 5). Section 6 concludes.

2. Background and related work

2.1 Resource exhaustion in conventional systems

Resource exhaustion has a long history as a first-class security concern. Mirkovic and Reiher (2004) catalogued distributed denial of service attacks and defenses by mechanism, and that work established the habit of treating availability as something to be engineered against, not merely provisioned for. The cloud era added an economic variant. Kelly, Glavin and Barrett (2021) described denial of wallet in serverless computing, where pay-per-use billing means an attacker need not take a service offline to harm its owner; forcing it to scale and to pay is enough to cause economic distress.

2.2 Exhaustion in machine learning systems

Machine learning has its own lineage. Shumailov et al. (2021) showed that carefully constructed "*sponge examples*" can drive up the energy and latency of neural network inference, establishing that input content, and not only input volume, determines cost. This is an example of an earlier paper where 'denial-of-wallet' attack vectors are discussed. For LLMs specifically, Geiping et al. (2024) mapped a broad space of adversarial objectives, denial of service among them, while Gao et al. (2024) demonstrated that poisoned training data can induce a model to generate until it reaches its output limit. Most relevant to modern deployments, Kumar et al. (2025) showed that decoy reasoning problems injected into retrieved content can substantially inflate the reasoning tokens consumed by reasoning-capable models, without changing what the user asked.

2.3 Cost-aware model selection

A parallel literature treats cost as an optimisation target. Chen, Zaharia and Zou (2023) proposed cascading and routing strategies that send queries to cheaper models where quality allows, and Ong et al. (2024) trained routers from preference data to the same end. These works motivate the model selector described in Section 5.3, although their goal is efficiency for honest traffic, not resilience against adversarial traffic.

2.4 The gap

Taken together, the literature shows that exhaustion techniques exist, that economic framing exists, and that routing mechanisms exist. What is less visible in public sources is a unified treatment that applies defense-in-depth to the whole session lifecycle and assigns ownership of the problem to security. TEA is offered as that treatment.

3. Defining the Token Exhaustion Attack

A Token Exhaustion Attack is the deliberate consumption of tokens, whether processed as input, retained as context, or generated as output including internal reasoning, with the objective of causing economic or operational harm to the operator of an LLM-backed system or to its legitimate users and in some cases to both.

Three features distinguish TEA from adjacent concepts. First, it is *adversarial and intentional*, which separates it from inefficient but honest use. Second, it targets the *token* as the unit of harm, so its effects can include degraded availability, inflated cost, displaced legitimate users, and pressure on shared infrastructure, often at once. Third, it can be mounted *without any policy violation*. Each prompt may be benign in content, and each request individually within limits, which is why content-focused guardrails are structurally blind to it.

3.1 Threat model

The attacker is assumed to be able to submit prompts through any legitimate interface, including a public chat endpoint, an API, a document upload that feeds retrieval, or a tool-using agent that can be steered by untrusted content. The attacker has no access to model weights or infrastructure. The attacker's advantage is *asymmetry*: a short, cheap input that provokes a long, expensive computation. The defender's difficulty is *attribution*. Because identical traffic may come from a genuinely demanding user, a defense that cannot tell the two apart will either miss attacks or penalise customers.

3.2 Why per-request limits are insufficient

Most deployed controls bound a single interaction, typically through a maximum input size, a maximum output length or a request rate. Three properties of LLM systems defeat this approach. First, cost accumulates across a session: because conversation history is generally resubmitted with each turn, a session of n turns of constant length processes a total input that grows roughly with the square of n. Second, the cost of attention in the transformer architecture itself grows quadratically with sequence length (Vaswani *et al.*, 2017), so context inflation compounds. Third, cost is not proportional to prompt length, a point established for neural networks generally by Shumailov *et al.* (2021) and for reasoning models by Kumar *et al.* (2025). A short prompt can be expensive and a long one cheap.

4. Attack vectors

The taxonomy below organises TEA techniques by the mechanism through which tokens are consumed. Six vectors derive from the structure of an LLM request. The seventh arises only when a defender introduces adaptive routing(AMS) and is included because defenses change the attack surface they protect.

Table 1. TEA vector taxonomy.

Vector	Mechanism	Principal signal
V1. Volume and padding	Copy-paste flooding or filler inside the prompt. A pure input attack.	Repetition, low information density
V2. Recursive prompting	Prompts that force the model to process its own output or to expand without bound before answering.	Instruction nesting depth, runaway generation
V3. Context window stuffing	Exploiting retrieval or long-context features to push duplicated, oversized or irrelevant content into context, inflating every request.	Abnormal or duplicate retrieval chunks
V4. Multi-turn accumulation ("slow bleed")	Individually small prompts that expand the conversation across many turns, exhausting the session budget rather than any one request.	Session-level growth in prompt and completion size
V5. Complexity inflation	Short prompts engineered to force expensive processing through ambiguity, bundled tasks or bait for extended reasoning.	Mismatch between prompt length and reasoning effort
V6. Output shape manipulation	Embedded instructions that leave reasoning unchanged but demand exhaustive lists, maximum detail or repeated variations.	Completion length against predicted ceiling
V7. Selector gaming	Engineering apparent complexity spikes and drops to force repeated, costly model switches.	Switch frequency, oscillating complexity score

4.1 Notes on selected vectors

Context stuffing (V3) is particularly relevant to agentic systems because the attacker need not interact with the model directly. Content placed in a document, web page or tool output is retrieved on the

attacker's behalf, and Kumar *et al.* (2025) demonstrate precisely this pattern for reasoning models. *Slow bleed (V4)* is the vector most likely to evade existing controls, since no single message is anomalous; it depends on a wide context window and sessions that are permitted to persist indefinitely.

Complexity inflation (V5) is the hardest to detect by inspection, because the prompt is unremarkable. Reasoning-capable models sharpen the problem. Their internal deliberation consumes tokens that are not part of the visible answer, and its length varies unpredictably with task difficulty. Routing a request to a reasoning model when the task does not need one therefore introduces an exhaustion risk that is independent of model size.

Selector gaming (V7) was identified analytically during design review and has not yet been validated experimentally. It is included now because it names an attack pattern whose mitigation differs materially from the others, and because it illustrates a general principle: any adaptive defense is itself a surface to be probed.

5. Mitigation architecture

The proposed architecture has three operational layers (input, cloud and output), an external model selector, and a cross-cutting requirement for monitoring and observability. This decomposition allows responsibility to be divided naturally among product security, red teams that test the controls, and product teams that build them in from the start.

5.1 Input controls

Input controls act before the model is invoked, which makes them the cheapest place to intervene. A historical baseline check compares prompt size against a rolling per-user or per-session norm. Repetition and loop detection scans for repeated n-grams, and a compression ratio or entropy check treats highly compressible input as a marker of low-information padding. A token-to-character ratio check flags zero-width characters, excess whitespace and unusual unicode sequences that inflate token counts without visible content. Instruction nesting depth counts stacked recursive directives. For retrieval pipelines, chunk-level checks detect duplicated or abnormally large content entering context. Finally, a session size accumulation check tracks whether prompts are growing steadily across a conversation, which is the principal defense against V4.

5.2 Output controls

Output controls address the fact that input inspection alone cannot predict cost. Actual completion length is compared with a ceiling predicted at prompt time. Repetition scoring and compression ratio checks are applied to generated text, since degenerate looping is a well-documented failure mode of autoregressive generation (Holtzman *et al.*, 2020) and is equally available to an attacker as a deliberate goal. Streaming checkpoint budgets examine token counts at fixed intervals during generation so that a runaway response can be terminated mid-stream, and tokens-per-second anomalies can reveal stalls or near-identical token emission. For reasoning models, reasoning tokens are tracked separately from answer tokens, because a blown-out reasoning trace is a distinct pattern from a blown-out answer. Session-level completion budgets catch turns that are individually acceptable but collectively excessive.

5.3 The Automated Model Selector

The Automated Model Selector (AMS) scores prompt complexity so that routing logic can match each conversation to an appropriately sized model, in the spirit of cost-aware routing (Chen, Zaharia and Zou, 2023; Ong *et al.*, 2024). The AMS evaluates four factors: expected reasoning depth, context size, ambiguity and predicted back-and-forth allowance. It emits a single numeric complexity score and nothing else.

This narrowness is a security decision, not a convenience. A large model that reasons its way to a routing decision would place a second unbounded cost in front of the first, and would itself be exposed to complexity inflation. By implementing the AMS as a small fine-tuned classifier or embedding-based scorer with a single numeric output, that attack surface is designed out instead of merely constrained. The choice of which model to mount is made downstream by the production team's routing logic. The score also supports a cross-check: a low-complexity score paired with high prompt length is itself a signal worth logging, and a persistently high reasoning-to-answer ratio for a task category indicates that routing thresholds need tuning.

Two caveats apply. Switching models mid-conversation can lose context, and the resulting token cost should be accepted as an unavoidable overhead and not engineered away. More importantly, the switching mechanism becomes the target of V7. The proposed mitigations are to rate-limit switch frequency and to require a sustained complexity signal across several turns before a swap is triggered. Confidence in these mitigations is lower than for the input and output controls, since V7 has been reasoned about but not tested.

5.4 Observability as a cross-cutting requirement

Monitoring should exist at a minimum of two stages, ideally input and output, so that defenders can see what was sent to the model, what came back and how the output was reached. This requirement carries a secondary benefit. Systems designed with TEA in mind must log enough to support detection, and that same logging supports audit, legal discovery and compliance decision making. The cost of defending against TEA is therefore partly recovered in governance value.

Table 2. Controls mapped to the vectors they principally address.

Layer	Representative controls	Vectors addressed
Input	Baseline check; repetition and entropy checks; unicode anomaly checks; nesting depth; retrieval chunk checks	V1, V2, V3
Session	Session size accumulation; session completion budgets	V4
Output	Predicted ceiling comparison; streaming checkpoints; output repetition scoring; reasoning-to-answer ratio	V2, V5, V6
AMS	Complexity scoring; length-versus-complexity cross-check; switch rate limiting	V5, V7

6. Conclusion

Token exhaustion is the denial of service problem of the LLM era, expressed in a currency that organisations have so far chosen to manage as a budget line. Treating it as a security attack vector costs little: it requires session-level thinking, detection at both ends of the model, a selector that cannot itself be exhausted, and the observability that good engineering ought to include anyway. The case for TEA does not depend on the claim that nobody has thought of this before. It depends on the claim that a risk with an attacker, an asymmetry and a victim deserves the same engineering discipline as every other risk with those properties. The next papers in this series examine detection rules in detail, the design and evaluation of the AMS, and the open question of distinguishing attack from legitimate demand.

Acknowledgements

The author thanks Jamie McGurk, Fergus Rea and Ciaran Chaney for their help in the research of this paper.

References

Chen, L., Zaharia, M. and Zou, J. (2023) *FrugalGPT: how to use large language models while reducing cost and improving performance*. arXiv:2305.05176. Available at: https://arxiv.org/abs/2305.05176.

Gao, K., Pang, T., Du, C., Yang, Y., Xia, S.-T. and Lin, M. (2024) *Denial-of-service poisoning attacks against large language models*. arXiv:2410.10760. Available at: https://arxiv.org/abs/2410.10760.

Geiping, J., Stein, A., Shu, M., Saifullah, K., Wen, Y. and Goldstein, T. (2024) *Coercing LLMs to do and reveal (almost) anything*. arXiv:2402.14020. Available at: https://arxiv.org/abs/2402.14020.

Holtzman, A., Buys, J., Du, L., Forbes, M. and Choi, Y. (2020) 'The curious case of neural text degeneration', in *International Conference on Learning Representations (ICLR 2020)*.

Kelly, D., Glavin, F.G. and Barrett, E. (2021) 'Denial of wallet: defining a looming threat to serverless computing', *Journal of Information Security and Applications*, 60, article 102843.

Kumar, A., Roh, J., Naseh, A., Karpinska, M., Iyyer, M., Houmansadr, A. and Bagdasarian, E. (2025) *OverThink: slowdown attacks on reasoning LLMs*. arXiv:2502.02542. Available at: https://arxiv.org/abs/2502.02542.

Mirkovic, J. and Reiher, P. (2004) 'A taxonomy of DDoS attack and DDoS defense mechanisms', *ACM SIGCOMM Computer Communication Review*, 34(2), pp. 39-53.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J.E., Kadous, M.W. and Stoica, I. (2024) *RouteLLM: learning to route LLMs with preference data*. arXiv:2406.18665. Available at: https://arxiv.org/abs/2406.18665.

OWASP (2025) *OWASP Top 10 for LLM Applications 2025: LLM10 Unbounded Consumption*. OWASP Gen AI Security Project. Available at: https://genai.owasp.org/.

Shumailov, I., Zhao, Y., Bates, D., Papernot, N., Mullins, R. and Anderson, R. (2021) 'Sponge examples: energy-latency attacks on neural networks', in *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, pp. 212-231.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.